

Lab 7 – Multiplayer Tic-Tac-Toe

OCSF & EventBus | Software Engineering, Spring 2025–2026

Submitters: 318535069, 206974073

1. Overview

A two-player networked Tic-Tac-Toe game built on the OCSF (Object Client-Server Framework). Two players connect to a server and play against each other; the server coordinates the match, tracks every move and keeps both players synchronized. The client GUI is built with JavaFX, and the client uses GreenRobot **EventBus** as a mediator (publish/subscribe) between the network layer and the user interface. Symbols (X / O) and the starting player are assigned randomly. While only one player is connected, that player waits until a second one joins.

2. Project structure

The project is a Maven multi-module build (the same skeleton as the course OCSF template):

- **entities** – shared, serializable messages exchanged over the socket: `Move`, `GameStartMessage`, `MoveMessage`, `GameOverMessage`.
- **server** – an OCSF `AbstractServer` (`SimpleServer`) plus a `Game` class that holds the board and the rules. The server is authoritative: it validates moves, detects win/draw and broadcasts state.
- **client** – a JavaFX app. `SimpleClient` (an OCSF `AbstractClient`) turns each server message into an EventBus event; `GameController` subscribes and updates the board on the JavaFX thread.

3. How it works (message flow)

- On connecting, the client sends "join". The first player receives *"waiting for opponent"* and waits.
- When a second player joins, the server creates a `Game`, randomly assigns X/O and the starting symbol, and sends each player a `GameStartMessage(symbol, starter)`.
- On a click the client sends a `Move(row, col)`. The server checks it is that player's turn and the cell is empty, applies it, then broadcasts a `MoveMessage(row, col, symbol, nextTurn)` to **both** players, so the two boards stay identical.
- When a line completes or the board fills, the server sends a `GameOverMessage(draw, winner)` to both players (You win / You lose / Draw).
- If a player disconnects mid-game, the opponent is told *"opponent disconnected"* and wins.

Because the server is the single source of truth and clients render only what it broadcasts, the two boards can never diverge. The EventBus mediator keeps the networking and the GUI loosely coupled: `SimpleClient` only *posts* events and never touches UI controls.

4. How to run

- Run **Maven install** in the parent project (`OCSFMediatorExample`).
- Start the **server**: server module, goal `exec:java` (main class `...server.App`). Default port 3000.
- Start the **client**: client module, goal `javafx:run`. Enable *Allow multiple instances* in the run configuration and launch it twice to get two players.

- In each client window enter the host and port and click **Connect**.

Running as JARs. `mvn clean install` produces `server-...-jar-with-dependencies.jar` and a shaded `client-...jar`, runnable with `java -jar`. The client pom auto-selects the matching JavaFX native libraries for the build machine (Windows / macOS Intel / macOS Apple Silicon / Linux).

Two computers. Start the server on one machine; on the client machine enter the server machine's IP address instead of localhost with the same port (see the "communication between 2 computers" note for firewall/IP setup).

5. Repository

Repository link: **<PASTE YOUR GITHUB CLASSROOM REPO LINK HERE>**

The two JAR files must be added to the repository (server jar and client jar).

6. Run on two different computers

Insert below a single photo of the two computers side by side, both running the game (one running the server + a client, the other running a client), as required by the submission instructions.

<INSERT PHOTO OF THE TWO COMPUTERS RUNNING THE GAME HERE>